

This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

Today 8:33 AM

Uploaded a file

i have a Python Pygame coin collector game. Here are my files Bug `update()` scores a coin every frame while the player overlaps it because coins are never removed. Fix it so each coin is collected exactly once, and explain the change.

Worked for 16s

Fixed. The bug was in `GameEngine.update()`:

Python

Run

```
collected = check_collection(self.player, self.coins)
for coin in collected:
    self.score += coin.value
```

The coin stayed inside `self.coins`, so on the next frame `check_collection()` found the same coin again. At 60 FPS, standing on one coin could add **60 points per second**.

Fix

After awarding points, remove all collected coins:

Python

Run

```
def update(self):
    collected = check_collection(self.player, self.coins)

    for coin in collected:
        self.score += coin.value
```

```
# Remove collected coins so they can only be scored once.  
self.coins = [coin for coin in self.coins if coin not in collected]
```

Now the flow is:

Player overlaps coin → score increases once → coin removed → coin is no longer drawn or detected.

I also packaged the corrected project:

[Download the fixed Coin Collector project](#)

just give updated file full content

Yep — here is the **full updated** `game_engine.py` **content**, with only the required fix applied:

Python

Run

```
"""  
GameEngine: owns the player and all coins.  
  
Starter version: one coin type, no obstacles, no timer yet.  
Collected coins are removed after being scored so each coin  
can only be collected once.  
"""  
  
import random  
import pygame  
  
from game.player import Player  
from game.coin import Coin  
from game.collection import check_collection  
from game.renderer import WIDTH, HEIGHT  
  
NUM_COINS = 6  
COIN_VALUE = 1  
  
class GameEngine:  
    def __init__(self):  
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)  
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]  
        self.score = 0  
  
    def _random_coin(self):
```

```

        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)
        return Coin(x=x, y=y, radius=12, value=COIN_VALUE)

def handle_input(self, keys_pressed):
    dx = dy = 0

    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed

    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed

    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed

    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed

    self.player.move(dx, dy, WIDTH, HEIGHT)

def update(self):
    collected = check_collection(self.player, self.coins)

    for coin in collected:
        self.score += coin.value

    # Remove collected coins so they cannot be scored again.
    self.coins = [coin for coin in self.coins if coin not in collected]

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.player,
        self.coins
    )

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

```

What changed?

Only this was added to `update()`:

Python

Run

```
self.coins = [coin for coin in self.coins if coin not in collected]
```

Previously, the collected coin remained in `self.coins`, so every frame:

```
overlap → +1 → overlap → +1 → overlap → +1 ...
```

Now:

```
overlap → +1 → coin removed → no more points
```

So each coin is collected exactly once.

Now add three coin types: bronze (1 point), silver (3), gold (5), each with its own colour. Make sure the right value is awarded. Give me the updated files.

Worked for 12s

Done. I updated the game to have:

-  **Bronze** → **1 point** → `(205, 127, 50)`
-  **Silver** → **3 points** → `(192, 192, 192)`
-  **Gold** → **5 points** → `(255, 215, 0)`
- Each coin is randomly assigned one of the three types.
- The coin's own `value` is used when scoring.
- Collected coins are removed, so they can only score **once**.
- All Python files pass syntax checking.

Updated `game/coin.py`

Python

Run

```
"""
Coin: a static collectible circle. Each coin has a value and colour.
"""

import pygame

class Coin:
    def __init__(self, x, y, radius=12, value=1, color=(205, 127, 50)):
        self.x = x
        self.y = y
```

```

        self.radius = radius
        self.value = value
        self.color = color

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius), int(self.y - self.radius),
            self.radius * 2, self.radius * 2,
        )

```

Updated `game/game_engine.py`

Python

Run

```

"""
GameEngine: owns the player and all coins.

There are three coin types:
- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

Collected coins are removed after scoring so each coin can only be
collected once.
"""

import random
import pygame

from game.player import Player
from game.coin import Coin
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6

# Coin type: (value, colour)
COIN_TYPES = [
    (1, (205, 127, 50)), # Bronze
    (3, (192, 192, 192)), # Silver
    (5, (255, 215, 0)), # Gold
]

class GameEngine:
    def __init__(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]
        self.score = 0

    def _random_coin(self):

```

```

x = random.randint(30, WIDTH - 30)
y = random.randint(30, HEIGHT - 30)

value, color = random.choice(COIN_TYPES)
return Coin(x=x, y=y, radius=12, value=value, color=color)

def handle_input(self, keys_pressed):
    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed
    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed
    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed
    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed
    self.player.move(dx, dy, WIDTH, HEIGHT)

def update(self):
    collected = check_collection(self.player, self.coins)

    # Award each collected coin's own value.
    for coin in collected:
        self.score += coin.value

    # Remove collected coins so they cannot be scored again.
    self.coins = [coin for coin in self.coins if coin not in collected]

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.player, self.coins)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))

```

`renderer.py`, `collection.py`, `player.py`, and `main.py` don't need changes because the renderer already uses `coin.color` and the collection logic already returns the actual `Coin` objects.

Full updated project:

Download the updated Coin Collector project

"Now add moving obstacles that bounce inside the play area. Touching one should cost a life (start with 3), with a short invulnerability window so one touch doesn't drain lives every frame. Show lives on screen.

Done. I added the moving obstacles and life system while keeping the existing three coin types and one-time collection behavior.

Changes

- **3 moving obstacles** are created.
- Obstacles move continuously and **bounce off all four play-area edges**.
- Player touching an obstacle loses **1 life**.
- Player starts with **3 lives**.
- A **1-second invulnerability window** starts after a hit, preventing one collision from draining multiple lives.
- **Lives: X** is displayed below the score.
- Lives cannot go below **0**.
- Coin scoring remains:
 - Bronze = **1**
 - Silver = **3**
 - Gold = **5**

Updated files

The project now contains:

```
coin-collector/  
├─ main.py  
├─ requirements.txt  
└─ game/  
    ├─ __init__.py  
    ├─ coin.py  
    ├─ collection.py  
    ├─ game_engine.py      ← updated  
    ├─ obstacle.py        ← NEW  
    ├─ player.py  
    └─ renderer.py        ← updated
```

Download the complete updated project

Download `06_coin_collector-obstacles.zip`

The code has also been syntax-checked successfully.

no please dont give updated zip just give those updated files full content

Absolutely. Only the files that need changing are below. Replace these **3 files**; everything else stays the same.

game/obstacle.py — **NEW**

Python

Run

```
"""
Moving obstacle that bounces inside the play area.
"""

import pygame

class Obstacle:
    def __init__(
        self,
        x,
        y,
        radius=16,
        dx=3,
        dy=2,
        color=(220, 60, 60),
    ):
        self.x = x
        self.y = y
        self.radius = radius
        self.dx = dx
        self.dy = dy
        self.color = color

    def update(self, width, height):
        self.x += self.dx
        self.y += self.dy

        # Bounce off the left and right edges.
        if self.x - self.radius <= 0:
            self.x = self.radius
            self.dx = abs(self.dx)

        elif self.x + self.radius >= width:
            self.x = width - self.radius
            self.dx = -abs(self.dx)

        # Bounce off the top and bottom edges.
        if self.y - self.radius <= 0:
            self.y = self.radius
            self.dy = abs(self.dy)

        elif self.y + self.radius >= height:
            self.y = height - self.radius
```

```

        self.dy = -abs(self.dy)

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius),
            int(self.y - self.radius),
            self.radius * 2,
            self.radius * 2,
        )

```

game/game_engine.py — UPDATED

Python

Run

```

"""
GameEngine: owns the player, coins, and moving obstacles.

Coin types:
- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

The player starts with 3 lives.
Touching an obstacle removes one life, followed by a short
invulnerability period so one collision cannot remove multiple lives.
"""

import random
import time
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
NUM_OBSTACLES = 3

STARTING_LIVES = 3
INVULNERABILITY_TIME = 1.0

# Coin type: (value, colour)
COIN_TYPES = [
    (1, (205, 127, 50)), # Bronze
    (3, (192, 192, 192)), # Silver
    (5, (255, 215, 0)), # Gold
]

```

```

class GameEngine:
    def __init__(self):
        self.player = Player(
            x=WIDTH / 2,
            y=HEIGHT / 2
        )

        self.coins = [
            self._random_coin()
            for _ in range(NUM_COINS)
        ]

        self.obstacles = [
            self._random_obstacle()
            for _ in range(NUM_OBSTACLES)
        ]

        self.score = 0

        # Life system
        self.lives = STARTING_LIVES
        self.invulnerable_until = 0.0

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)

        value, color = random.choice(COIN_TYPES)

        return Coin(
            x=x,
            y=y,
            radius=12,
            value=value,
            color=color
        )

    def _random_obstacle(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)

        # Random movement speed.
        dx = random.choice([-3, -2, 2, 3])
        dy = random.choice([-3, -2, 2, 3])

        return Obstacle(
            x=x,
            y=y,
            radius=16,
            dx=dx,

```

```

        dy=dy,
        color=(220, 60, 60)
    )

def handle_input(self, keys_pressed):
    dx = dy = 0

    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed

    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed

    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed

    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed

    self.player.move(
        dx,
        dy,
        WIDTH,
        HEIGHT
    )

def update(self):
    # -----
    # Update moving obstacles
    # -----
    for obstacle in self.obstacles:
        obstacle.update(WIDTH, HEIGHT)

    # -----
    # Coin collection
    # -----
    collected = check_collection(
        self.player,
        self.coins
    )

    for coin in collected:
        self.score += coin.value

    # Remove collected coins so they cannot
    # be scored again.
    self.coins = [
        coin
        for coin in self.coins
        if coin not in collected
    ]

```

```

# -----
# Obstacle collision
# -----
current_time = time.monotonic()

# Only allow damage if the player is
# not currently invulnerable.
if current_time >= self.invulnerable_until:
    player_rect = self.player.get_rect()

    for obstacle in self.obstacles:
        if player_rect.colliderect(
            obstacle.get_rect()
        ):
            self.lives -= 1

            # Start the invulnerability period.
            self.invulnerable_until = (
                current_time + INVULNERABILITY_TIME
            )

            # Only one life can be lost per update.
            break

# Prevent negative lives.
if self.lives < 0:
    self.lives = 0

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.player,
        self.coins,
        self.obstacles
    )

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 40)
    )

```

Python

Run

```
"""
Renderer: draws the game scene.
"""

import pygame

WIDTH = 800
HEIGHT = 600

BACKGROUND_COLOR = (30, 30, 30)
PLAYER_COLOR = (50, 150, 255)

def draw_scene(
    surface,
    player,
    coins,
    obstacles=None
):
    surface.fill(BACKGROUND_COLOR)

    # Draw coins.
    for coin in coins:
        pygame.draw.circle(
            surface,
            coin.color,
            (int(coin.x), int(coin.y)),
            coin.radius
        )

    # Draw moving obstacles.
    if obstacles is not None:
        for obstacle in obstacles:
            pygame.draw.circle(
                surface,
                obstacle.color,
                (int(obstacle.x), int(obstacle.y)),
                obstacle.radius
            )

    # Draw player.
    pygame.draw.circle(
        surface,
        PLAYER_COLOR,
        (int(player.x), int(player.y)),
        player.radius
    )
```

```
def draw_text(surface, font, text, position):
    text_surface = font.render(
        text,
        True,
        (255, 255, 255)
    )

    surface.blit(
        text_surface,
        position
    )
```

One important detail

The invulnerability logic is:

Python

Run

```
self.invulnerable_until = current_time + 1.0
```

So after getting hit:

```
Hit obstacle
↓
Lives - 1
↓
1 second invulnerable
↓
Can touch obstacle without losing another life
↓
After 1 second → collision can cost another life
```

And because the collision check uses:

Python

Run

```
player_rect.collidect(obstacle.get_rect())
```

the player only loses **one life per collision event**, rather than losing a life every frame while overlapping.

Now add a 30-second countdown shown on screen. The round ends when time runs out or lives hit zero, shows the

final score, and lets me press R to restart with score, lives and timer reset

Worked for 9s

Yep — for this change, replace **these 3 files**. Your existing `coin.py`, `collection.py`, `player.py`, and `obstacle.py` stay unchanged from the previous version.

`game/game_engine.py`

Python

Run

```
"""
GameEngine: owns the player, coins, and moving obstacles.

Coin types:
- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

The player starts with 3 lives and has 30 seconds per round.
The round ends when the timer reaches zero or lives reach zero.
Press R to restart the round.
"""

import random
import time
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
NUM_OBSTACLES = 3

STARTING_LIVES = 3
ROUND_TIME = 30.0
INVULNERABILITY_TIME = 1.0

COIN_TYPES = [
    (1, (205, 127, 50)), # Bronze
    (3, (192, 192, 192)), # Silver
    (5, (255, 215, 0)), # Gold
]
```

```

class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Reset the entire round."""
        self.player = Player(
            x=WIDTH / 2,
            y=HEIGHT / 2
        )

        self.coins = [
            self._random_coin()
            for _ in range(NUM_COINS)
        ]

        self.obstacles = [
            self._random_obstacle()
            for _ in range(NUM_OBSTACLES)
        ]

        self.score = 0
        self.lives = STARTING_LIVES

        # Time at which the player can next be damaged.
        self.invulnerable_until = 0.0

        # Start the 30-second countdown.
        self.start_time = time.monotonic()

        self.game_over = False

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(70, HEIGHT - 30)

        value, color = random.choice(COIN_TYPES)

        return Coin(
            x=x,
            y=y,
            radius=12,
            value=value,
            color=color
        )

    def _random_obstacle(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(70, HEIGHT - 30)

        dx = random.choice([-3, -2, 2, 3])
        dy = random.choice([-3, -2, 2, 3])

```

```

        return Obstacle(
            x=x,
            y=y,
            radius=16,
            dx=dx,
            dy=dy,
            color=(220, 60, 60)
        )

def handle_input(self, keys_pressed):
    # Do not allow movement after the round ends.
    if self.game_over:
        return

    dx = 0
    dy = 0

    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed

    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed

    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed

    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed

    self.player.move(
        dx,
        dy,
        WIDTH,
        HEIGHT
    )

def update(self):
    # Do nothing once the round has ended.
    if self.game_over:
        return

    current_time = time.monotonic()

    # -----
    # Countdown timer
    # -----
    elapsed_time = current_time - self.start_time
    remaining_time = max(0.0, ROUND_TIME - elapsed_time)

    if remaining_time <= 0:
        self.game_over = True

```

```

        return

# -----
# Update moving obstacles
# -----
for obstacle in self.obstacles:
    obstacle.update(WIDTH, HEIGHT)

# -----
# Coin collection
# -----
collected = check_collection(
    self.player,
    self.coins
)

for coin in collected:
    self.score += coin.value

# Remove collected coins so they
# cannot be scored again.
self.coins = [
    coin
    for coin in self.coins
    if coin not in collected
]

# -----
# Obstacle collision
# -----
if current_time >= self.invulnerable_until:
    player_rect = self.player.get_rect()

    for obstacle in self.obstacles:
        if player_rect.colliderect(
            obstacle.get_rect()
        ):
            self.lives -= 1

            # Give the player one second
            # of invulnerability.
            self.invulnerable_until = (
                current_time + INVULNERABILITY_TIME
            )

            break

# -----
# Check for game over
# -----
if self.lives <= 0:
    self.lives = 0

```

```

        self.game_over = True

def get_remaining_time(self):
    """Return the number of seconds remaining."""
    if self.game_over:
        return max(
            0.0,
            ROUND_TIME - (
                time.monotonic() - self.start_time
            )
        )

    return max(
        0.0,
        ROUND_TIME - (
            time.monotonic() - self.start_time
        )
    )

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.player,
        self.coins,
        self.obstacles
    )

    remaining_time = self.get_remaining_time()

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 40)
    )

    renderer.draw_text(
        surface,
        font,
        f"Time: {int(remaining_time + 0.999)}",
        (10, 70)
    )

```

```
# -----  
# Game-over screen  
# -----  
if self.game_over:  
    renderer.draw_game_over(  
        surface,  
        font,
```

game/renderer.py

Python

Run

```
"""  
renderer: all pygame drawing lives here.  
"""  
  
import pygame  
  
WIDTH, HEIGHT = 700, 500  
WINDOW_SIZE = (WIDTH, HEIGHT)  
  
COLOR_BG = (35, 45, 35)  
COLOR_PLAYER = (80, 180, 255)  
COLOR_TEXT = (255, 255, 255)  
  
def draw_scene(surface, player, coins, obstacles=None):  
    surface.fill(COLOR_BG)  
  
    # Draw coins.  
    for coin in coins:  
        pygame.draw.circle(  
            surface,  
            coin.color,  
            (int(coin.x), int(coin.y)),  
            coin.radius  
        )  
  
    # Draw moving obstacles.  
    if obstacles is not None:  
        for obstacle in obstacles:  
            pygame.draw.circle(  
                surface,  
                obstacle.color,  
                (int(obstacle.x), int(obstacle.y)),  
                obstacle.radius  
            )  
  
    # Draw player.
```



```

game_over_surface = game_over_font.render(
    "GAME OVER",
    True,
    (255, 220, 80)
)

score_surface = score_font.render(
    f"Final Score: {score}",
    True,
    (255, 255, 255)
)

restart_surface = restart_font.render(
    "Press R to restart",
    True,
    (200, 200, 200)
)

center_x = surface.get_width() // 2

surface.blit(
    game_over_surface,
    game_over_surface.get_rect(
        center=(center_x, 190)
    )
)

surface.blit(
    score_surface,
    score_surface.get_rect(
        center=(center_x, 245)
    )
)

surface.blit(
    restart_surface,
    restart_surface.get_rect(
        center=(center_x, 300)
    )
)

```

main.py

Python

Run

```

"""
Coin Collector

Controls:
    Arrow keys - Move

```



```
pygame.quit()

if __name__ == "__main__":
    main()
```

What the new round flow does

```
Start
↓
30 seconds
↓
Collect coins → score increases
↓
Hit obstacle → lose 1 life
↓
1 second invulnerability
↓
Timer reaches 0 OR lives reach 0
↓
GAME OVER
↓
Final Score shown
↓
Press R
↓
New 30-second round
Score = 0
Lives = 3
Timer = 30
Coins + obstacles regenerated
```

The timer uses `time.monotonic()`, so it counts actual elapsed time rather than losing/gaining time based on the frame rate.

In my Pygame coin collector, the countdown timer keeps decreasing on the game-over screen when the round ends because lives hit zero. `get_remaining_time()` in `game_engine.py` recomputes from `time.monotonic()` even after game over. Fix it so the remaining time is frozen when the round ends, and show the code changes.

Yes. The issue is that `get_remaining_time()` always calculates:

Python

Run

```
ROUND_TIME - (time.monotonic() - self.start_time)
```

even after `self.game_over` becomes `True`.

1. Add a frozen timer value in `reset()`

Python

Run

```
def reset(self):
    self.player = Player(
        x=WIDTH / 2,
        y=HEIGHT / 2
    )

    self.coins = [
        self._random_coin()
        for _ in range(NUM_COINS)
    ]

    self.obstacles = [
        self._random_obstacle()
        for _ in range(NUM_OBSTACLES)
    ]

    self.score = 0
    self.lives = STARTING_LIVES

    self.invulnerable_until = 0.0

    self.start_time = time.monotonic()

    # Current remaining time.
    self.remaining_time = ROUND_TIME

    self.game_over = False
```

2. Freeze the time when the round ends

Change the timer section in `update()` to:

Python

Run

```
elapsed_time = current_time - self.start_time
self.remaining_time = max(0.0, ROUND_TIME - elapsed_time)

if self.remaining_time <= 0:
    self.remaining_time = 0.0
```

```
self.game_over = True
```

Then change the lives game-over section to:

Python

Run

```
if self.lives <= 0:
    self.lives = 0

    # Freeze the timer at the exact moment
    # the player loses the final life.
    self.remaining_time = max(
        0.0,
        ROUND_TIME - (current_time - self.start_time)
    )

    self.game_over = True
```

3. Replace `get_remaining_time()`

The important fix is to **stop recalculating after game over**:

Python

Run

```
def get_remaining_time(self):
    if self.game_over:
        return self.remaining_time

    current_time = time.monotonic()

    elapsed_time = current_time - self.start_time

    self.remaining_time = max(
        0.0,
        ROUND_TIME - elapsed_time
    )

    return self.remaining_time
```

4. Your `draw()` can stay the same

Python

Run

```
remaining_time = self.get_remaining_time()

renderer.draw_text(
    surface,
    font,
    f"Time: {int(remaining_time + 0.999)}",
```

(10, 70)

Now, if the player has **8.4 seconds remaining** when their third life is lost:

```
Lives → 0
Timer → 8 seconds
GAME OVER

Press R
↓
Timer → 30 seconds
Lives → 3
Score → 0
```

The timer will remain frozen at that value throughout the game-over screen.

just give full updated file

Here is the **full updated** `game/game_engine.py` with the timer-freezing fix included:

Python

Run

```
"""
GameEngine: owns the player, coins, and moving obstacles.

Coin types:
- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

The player starts with 3 lives and has 30 seconds per round.
The round ends when the timer reaches zero or lives reach zero.
Press R to restart the round.
"""

import random
import time
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
NUM_OBSTACLES = 3
```

```

STARTING_LIVES = 3
ROUND_TIME = 30.0
INVULNERABILITY_TIME = 1.0

COIN_TYPES = [
    (1, (205, 127, 50)),    # Bronze
    (3, (192, 192, 192)),  # Silver
    (5, (255, 215, 0)),    # Gold
]

class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Reset the entire round."""

        self.player = Player(
            x=WIDTH / 2,
            y=HEIGHT / 2
        )

        self.coins = [
            self._random_coin()
            for _ in range(NUM_COINS)
        ]

        self.obstacles = [
            self._random_obstacle()
            for _ in range(NUM_OBSTACLES)
        ]

        self.score = 0
        self.lives = STARTING_LIVES

        # Time until which the player is invulnerable.
        self.invulnerable_until = 0.0

        # Start the countdown.
        self.start_time = time.monotonic()

        # Store the current remaining time.
        # This is important because it allows the timer
        # to remain frozen after game over.
        self.remaining_time = ROUND_TIME

        self.game_over = False

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)

```

```

y = random.randint(70, HEIGHT - 30)

value, color = random.choice(COIN_TYPES)

return Coin(
    x=x,
    y=y,
    radius=12,
    value=value,
    color=color
)

def _random_obstacle(self):
    x = random.randint(30, WIDTH - 30)
    y = random.randint(70, HEIGHT - 30)

    dx = random.choice([-3, -2, 2, 3])
    dy = random.choice([-3, -2, 2, 3])

    return Obstacle(
        x=x,
        y=y,
        radius=16,
        dx=dx,
        dy=dy,
        color=(220, 60, 60)
    )

def handle_input(self, keys_pressed):
    # Do not allow movement after game over.
    if self.game_over:
        return

    dx = 0
    dy = 0

    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed

    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed

    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed

    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed

    self.player.move(
        dx,
        dy,
        WIDTH,

```

```

        HEIGHT
    )

def update(self):
    # Do nothing after game over.
    if self.game_over:
        return

    current_time = time.monotonic()

    # -----
    # Countdown timer
    # -----
    elapsed_time = current_time - self.start_time

    self.remaining_time = max(
        0.0,
        ROUND_TIME - elapsed_time
    )

    # Timer reached zero.
    if self.remaining_time <= 0:
        self.remaining_time = 0.0
        self.game_over = True
        return

    # -----
    # Update moving obstacles
    # -----
    for obstacle in self.obstacles:
        obstacle.update(WIDTH, HEIGHT)

    # -----
    # Coin collection
    # -----
    collected = check_collection(
        self.player,
        self.coins
    )

    for coin in collected:
        self.score += coin.value

    # Remove collected coins so they
    # cannot be scored again.
    self.coins = [
        coin
        for coin in self.coins
        if coin not in collected
    ]

    # -----

```

```

# Obstacle collision
# -----
if current_time >= self.invulnerable_until:
    player_rect = self.player.get_rect()

    for obstacle in self.obstacles:
        if player_rect.colliderect(
            obstacle.get_rect()
        ):
            self.lives -= 1

            # Give the player one second
            # of invulnerability.
            self.invulnerable_until = (
                current_time + INVULNERABILITY_TIME
            )

            break

# -----
# Check for game over
# -----
if self.lives <= 0:
    self.lives = 0

    # IMPORTANT:
    # Save the exact remaining time when
    # the final life is lost.
    self.remaining_time = max(
        0.0,
        ROUND_TIME - (
            current_time - self.start_time
        )
    )

    self.game_over = True

def get_remaining_time(self):
    """
    Return the remaining round time.

    Once the game is over, return the stored value
    instead of calculating from time.monotonic().
    This keeps the timer frozen on the game-over screen.
    """

    if self.game_over:
        return self.remaining_time

    current_time = time.monotonic()

    elapsed_time = current_time - self.start_time

```

```

        self.remaining_time = max(
            0.0,
            ROUND_TIME - elapsed_time
        )

    return self.remaining_time

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.player,
        self.coins,
        self.obstacles
    )

    remaining_time = self.get_remaining_time()

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 40)
    )

    renderer.draw_text(
        surface,
        font,
        f"Time: {int(remaining_time + 0.999)}",
        (10, 70)
    )

    # -----
    # Game-over screen
    # -----
    if self.game_over:
        renderer.draw_game_over(
            surface,
            font,
            self.score

```

The key change is that `self.remaining_time` is now **stored when game over happens**, and `get_remaining_time()` returns that stored value instead of continuing to calculate

elapsed time.

Right now collected coins are removed and never come back, so after 6 coins the screen is empty. Make a new random coin (random type, not overlapping an obstacle) spawn each time one is collected, so there are always 6 coins on screen. Keep each coin scoring exactly once.

Replace your `game/game_engine.py` with this full version. Everything else can stay unchanged.

Python

Run

```
"""
GameEngine: owns the player, coins, and moving obstacles.

Coin types:
- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

There are always 6 coins on screen.
When a coin is collected, a new random coin is spawned
at a position that does not overlap an obstacle.

Each individual coin can only be collected once.
"""

import random
import time
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
NUM_OBSTACLES = 3

STARTING_LIVES = 3
ROUND_TIME = 30.0
INVULNERABILITY_TIME = 1.0

COIN_RADIUS = 12
```



```

        if distance_squared < minimum_distance ** 2:
            return True

    return False

def _random_obstacle(self):
    x = random.randint(
        30,
        WIDTH - 30
    )

    y = random.randint(
        70,
        HEIGHT - 30
    )

    dx = random.choice(
        [-3, -2, 2, 3]
    )

    dy = random.choice(
        [-3, -2, 2, 3]
    )

    return Obstacle(
        x=x,
        y=y,
        radius=16,
        dx=dx,
        dy=dy,
        color=(220, 60, 60)
    )

def handle_input(self, keys_pressed):
    # Don't allow movement after game over.
    if self.game_over:
        return

    dx = 0
    dy = 0

    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed

    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed

    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed

    if keys_pressed[pygame.K_RIGHT]:

```

```

        dx += self.player.speed

    self.player.move(
        dx,
        dy,
        WIDTH,
        HEIGHT
    )

def update(self):
    # Do nothing after game over.
    if self.game_over:
        return

    current_time = time.monotonic()

    # -----
    # Countdown timer
    # -----

    elapsed_time = (
        current_time -
        self.start_time
    )

    self.remaining_time = max(
        0.0,
        ROUND_TIME - elapsed_time
    )

    # Timer reached zero.
    if self.remaining_time <= 0:
        self.remaining_time = 0.0
        self.game_over = True
        return

    # -----
    # Update moving obstacles
    # -----

    for obstacle in self.obstacles:
        obstacle.update(
            WIDTH,
            HEIGHT
        )

    # -----
    # Coin collection
    # -----

    collected = check_collection(
        self.player,

```

```

        self.coins
    )

    for coin in collected:
        # Award this coin exactly once.
        self.score += coin.value

    if collected:
        # Remove the collected coins first.
        self.coins = [
            coin
            for coin in self.coins
            if coin not in collected
        ]

        # Spawn exactly one replacement for
        # every collected coin.
        for _ in collected:
            self.coins.append(
                self._random_coin()
            )

    # -----
    # Obstacle collision
    # -----

    if current_time >= self.invulnerable_until:
        player_rect = self.player.get_rect()

        for obstacle in self.obstacles:
            if player_rect.colliderect(
                obstacle.get_rect()
            ):
                self.lives -= 1

                # One second of invulnerability.
                self.invulnerable_until = (
                    current_time +
                    INVULNERABILITY_TIME
                )

                break

    # -----
    # Check for game over
    # -----

    if self.lives <= 0:
        self.lives = 0

        # Freeze timer at the moment the final
        # life is lost.

```

```

        self.remaining_time = max(
            0.0,
            ROUND_TIME - (
                current_time -
                self.start_time
            )
        )

        self.game_over = True

def get_remaining_time(self):
    """
    Return remaining time.

    When the game is over, return the stored value
    so the timer stays frozen.
    """

    if self.game_over:
        return self.remaining_time

    current_time = time.monotonic()

    elapsed_time = (
        current_time -
        self.start_time
    )

    self.remaining_time = max(
        0.0,
        ROUND_TIME - elapsed_time
    )

    return self.remaining_time

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.player,
        self.coins,
        self.obstacles
    )

    remaining_time = (
        self.get_remaining_time()
    )

    renderer.draw_text(
        surface,
        font,

```

```

        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 40)
    )

    renderer.draw_text(
        surface,
        font,
        f"Time: {int(remaining_time + 0.999)}",
        (10, 70)
    )

    # -----
    # Game-over screen
    # -----

    if self.game_over:
        renderer.draw_game_over(
            surface,

```

What changed

Previously:

Python

Run

```

self.coins = [
    coin for coin in self.coins
    if coin not in collected
]

```

simply reduced the number of coins.

Now, immediately after removing collected coins:

Python

Run

```

for _ in collected:
    self.coins.append(self._random_coin())

```

a replacement is created for **every coin collected**.

So if the player collects 2 coins in the same frame:

```
6 coins
↓
2 collected
↓
score += both values
↓
remove 2
↓
spawn 2 new random coins
↓
6 coins again
```

`_random_coin()` also checks every obstacle before accepting the new position, so replacement coins won't spawn overlapping an obstacle.

Right now collected coins are removed and never come back, so after 6 coins the screen is empty. Make a new random coin (random type, not overlapping an obstacle) spawn each time one is collected, so there are always 6 coins on screen. Keep each coin scoring exactly once

Replace `game/game_engine.py` with this version. It keeps exactly **6 coins** on screen, gives each coin its own random type/value, avoids spawning coins on obstacles, and each coin scores only once.

Python

Run

```
"""
GameEngine: owns the player, coins, and moving obstacles.

Coin types:
- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

There are always 6 coins on screen.
When a coin is collected, a new random coin is spawned
at a position that does not overlap an obstacle.

Each coin is scored exactly once.
"""

import random
import time
import pygame
```

```

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
NUM_OBSTACLES = 3

STARTING_LIVES = 3
ROUND_TIME = 30.0
INVULNERABILITY_TIME = 1.0

COIN_RADIUS = 12

COIN_TYPES = [
    (1, (205, 127, 50)),    # Bronze
    (3, (192, 192, 192)),  # Silver
    (5, (255, 215, 0)),    # Gold
]

class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Reset the entire round."""

        self.player = Player(
            x=WIDTH / 2,
            y=HEIGHT / 2
        )

        self.score = 0
        self.lives = STARTING_LIVES

        self.invulnerable_until = 0.0

        self.start_time = time.monotonic()
        self.remaining_time = ROUND_TIME

        self.game_over = False

        # Create obstacles first so that coins
        # can avoid them when spawning.
        self.obstacles = [
            self._random_obstacle()
            for _ in range(NUM_OBSTACLES)
        ]

```

```

        # Always start with exactly 6 coins.
        self.coins = [
            self._random_coin()
            for _ in range(NUM_COINS)
        ]

def _random_coin(self):
    """
    Create a random coin that does not overlap
    any obstacle.
    """

    for _ in range(100):
        x = random.randint(
            COIN_RADIUS,
            WIDTH - COIN_RADIUS
        )

        y = random.randint(
            70 + COIN_RADIUS,
            HEIGHT - COIN_RADIUS
        )

        # Randomly choose Bronze, Silver, or Gold.
        value, color = random.choice(COIN_TYPES)

        candidate = Coin(
            x=x,
            y=y,
            radius=COIN_RADIUS,
            value=value,
            color=color
        )

        if not self._coin_overlaps_obstacle(candidate):
            return candidate

    # Fallback position if 100 attempts fail.
    value, color = random.choice(COIN_TYPES)

    return Coin(
        x=COIN_RADIUS + 5,
        y=75,
        radius=COIN_RADIUS,
        value=value,
        color=color
    )

def _coin_overlaps_obstacle(self, coin):
    """Check whether a coin overlaps any obstacle."""

```



```

        dy = 0

        if keys_pressed[pygame.K_UP]:
            dy -= self.player.speed

        if keys_pressed[pygame.K_DOWN]:
            dy += self.player.speed

        if keys_pressed[pygame.K_LEFT]:
            dx -= self.player.speed

        if keys_pressed[pygame.K_RIGHT]:
            dx += self.player.speed

        self.player.move(
            dx,
            dy,
            WIDTH,
            HEIGHT
        )

    def update(self):
        if self.game_over:
            return

        current_time = time.monotonic()

        # -----
        # Countdown timer
        # -----

        elapsed_time = (
            current_time - self.start_time
        )

        self.remaining_time = max(
            0.0,
            ROUND_TIME - elapsed_time
        )

        if self.remaining_time <= 0:
            self.remaining_time = 0.0
            self.game_over = True
            return

        # -----
        # Update obstacles
        # -----

        for obstacle in self.obstacles:
            obstacle.update(
                WIDTH,

```

```

        HEIGHT
    )

# -----
# Collect coins
# -----

collected = check_collection(
    self.player,
    self.coins
)

# Award each collected coin once.
for coin in collected:
    self.score += coin.value

if collected:
    # Remove collected coins.
    self.coins = [
        coin
        for coin in self.coins
        if coin not in collected
    ]

    # Spawn one replacement for every
    # coin that was collected.
    for _ in collected:
        self.coins.append(
            self._random_coin()
        )

# -----
# Obstacle collision
# -----

if current_time >= self.invulnerable_until:
    player_rect = self.player.get_rect()

    for obstacle in self.obstacles:
        if player_rect.colliderect(
            obstacle.get_rect()
        ):
            self.lives -= 1

            self.invulnerable_until = (
                current_time +
                INVULNERABILITY_TIME
            )

            break

# -----

```

```

# Lives game over
# -----

if self.lives <= 0:
    self.lives = 0

    # Freeze timer at game over.
    self.remaining_time = max(
        0.0,
        ROUND_TIME - (
            current_time -
            self.start_time
        )
    )

    self.game_over = True

def get_remaining_time(self):
    """
    Return remaining time.

    Once the round ends, the stored value is returned
    so the timer stays frozen.
    """

    if self.game_over:
        return self.remaining_time

    current_time = time.monotonic()

    elapsed_time = (
        current_time - self.start_time
    )

    self.remaining_time = max(
        0.0,
        ROUND_TIME - elapsed_time
    )

    return self.remaining_time

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.player,
        self.coins,
        self.obstacles
    )

    remaining_time = (

```

```

        self.get_remaining_time()
    )

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 40)
    )

    renderer.draw_text(
        surface,
        font,
        f"Time: {int(remaining_time + 0.999)}",
        (10, 70)
    )

    if self.game_over:
        renderer.draw_game_over(
            surface,
            font,

```

The important part is:

Python

Run

```

for coin in collected:
    self.score += coin.value

self.coins = [
    coin for coin in self.coins
    if coin not in collected
]

for _ in collected:
    self.coins.append(self._random_coin())

```

So **collect** → **score once** → **remove old coin** → **spawn replacement**, keeping the number of coins at **6 continuously**.

Obstacles sometimes spawn on top of the player at the start of a round and cost a life instantly. Make

`_random_obstacle` spawn at least 120 pixels away from the player's starting position

Replace the `_random_obstacle()` method in `game_engine.py` with this:

Python

Run

```
def _random_obstacle(self):
    """
    Create an obstacle at least 120 pixels away from
    the player's starting position.
    """

    player_x = self.player.x
    player_y = self.player.y

    minimum_distance = 120

    for _ in range(100):
        x = random.randint(
            30,
            WIDTH - 30
        )

        y = random.randint(
            70,
            HEIGHT - 30
        )

        dx = x - player_x
        dy = y - player_y

        distance = (dx * dx + dy * dy) ** 0.5

        if distance >= minimum_distance:
            return Obstacle(
                x=x,
                y=y,
                radius=16,
                dx=random.choice([-3, -2, 2, 3]),
                dy=random.choice([-3, -2, 2, 3]),
                color=(220, 60, 60)
            )

    # Fallback: place the obstacle away from the player.
    return Obstacle(
        x=30,
        y=70,
        radius=16,
```

```

        dx=random.choice([-3, -2, 2, 3]),
        dy=random.choice([-3, -2, 2, 3]),
        color=(220, 60, 60)
    )

```

The important check is:

Python

Run

```

distance = (dx * dx + dy * dy) ** 0.5

if distance >= 120:

```

Since `self.player` is created **before** the obstacles in `reset()`:

Python

Run

```

self.player = Player(
    x=WIDTH / 2,
    y=HEIGHT / 2
)

self.obstacles = [
    self._random_obstacle()
    for _ in range(NUM_OBSTACLES)
]

```

`_random_obstacle()` can safely use the player's starting position.

This guarantees that newly created obstacles are at least **120 pixels from the player's starting position**, preventing an immediate collision at the beginning of a round.

"""

GameEngine: owns the player, coins, and moving obstacles.

Coin types:

- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

There are always 6 coins on screen.

When a coin is collected, a new random coin is spawned

Show more

You have an **indentation error** in `_random_obstacle()`. The docstring and its entire body are currently not indented inside the method, which also causes `update()` to accidentally become nested inside `_random_obstacle()`.

Here is the **full corrected** `game_engine.py`:

Python

Run

```
"""
GameEngine: owns the player, coins, and moving obstacles.

Coin types:
- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

There are always 6 coins on screen.
When a coin is collected, a new random coin is spawned
at a position that does not overlap an obstacle.

Each coin is scored exactly once.
"""

import random
import time
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
NUM_OBSTACLES = 3

STARTING_LIVES = 3
ROUND_TIME = 30.0
INVULNERABILITY_TIME = 1.0

COIN_RADIUS = 12

COIN_TYPES = [
    (1, (205, 127, 50)), # Bronze
    (3, (192, 192, 192)), # Silver
    (5, (255, 215, 0)), # Gold
]
```

```

class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Reset the entire round."""

        self.player = Player(
            x=WIDTH / 2,
            y=HEIGHT / 2
        )

        self.score = 0
        self.lives = STARTING_LIVES

        self.invulnerable_until = 0.0

        self.start_time = time.monotonic()
        self.remaining_time = ROUND_TIME

        self.game_over = False

        # Create obstacles first so that coins
        # can avoid them when spawning.
        self.obstacles = [
            self._random_obstacle()
            for _ in range(NUM_OBSTACLES)
        ]

        # Always start with exactly 6 coins.
        self.coins = [
            self._random_coin()
            for _ in range(NUM_COINS)
        ]

    def _random_coin(self):
        """
        Create a random coin that does not overlap
        any obstacle.
        """

        for _ in range(100):
            x = random.randint(
                COIN_RADIUS,
                WIDTH - COIN_RADIUS
            )

            y = random.randint(
                70 + COIN_RADIUS,
                HEIGHT - COIN_RADIUS
            )

```



```

if self.game_over:
    return

current_time = time.monotonic()

# -----
# Countdown timer
# -----

elapsed_time = (
    current_time - self.start_time
)

self.remaining_time = max(
    0.0,
    ROUND_TIME - elapsed_time
)

if self.remaining_time <= 0:
    self.remaining_time = 0.0
    self.game_over = True
    return

# -----
# Update obstacles
# -----

for obstacle in self.obstacles:
    obstacle.update(
        WIDTH,
        HEIGHT
    )

# -----
# Collect coins
# -----

collected = check_collection(
    self.player,
    self.coins
)

# Award each collected coin exactly once.
for coin in collected:
    self.score += coin.value

if collected:
    # Remove collected coins.
    self.coins = [
        coin
        for coin in self.coins
        if coin not in collected
    ]

```

```

    ]

    # Spawn one replacement for every
    # coin that was collected.
    for _ in collected:
        self.coins.append(
            self._random_coin()
        )

    # -----
    # Obstacle collision
    # -----

    if current_time >= self.invulnerable_until:
        player_rect = self.player.get_rect()

        for obstacle in self.obstacles:
            if player_rect.colliderect(
                obstacle.get_rect()
            ):
                self.lives -= 1

                self.invulnerable_until = (
                    current_time +
                    INVULNERABILITY_TIME
                )

                break

    # -----
    # Lives game over
    # -----

    if self.lives <= 0:
        self.lives = 0

        # Freeze timer at game over.
        self.remaining_time = max(
            0.0,
            ROUND_TIME - (
                current_time -
                self.start_time
            )
        )

        self.game_over = True

def get_remaining_time(self):
    """
    Return remaining time.

    Once the round ends, the stored value is returned

```

```

        so the timer stays frozen.
        """

        if self.game_over:
            return self.remaining_time

        current_time = time.monotonic()

        elapsed_time = (
            current_time - self.start_time
        )

        self.remaining_time = max(
            0.0,
            ROUND_TIME - elapsed_time
        )

        return self.remaining_time

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.player,
        self.coins,
        self.obstacles
    )

    remaining_time = (
        self.get_remaining_time()
    )

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 40)
    )

    renderer.draw_text(
        surface,
        font,
        f"Time: {int(remaining_time + 0.999)}",
        (10, 70)
    )

```

```
)

if self.game_over:
    renderer.draw_game_over(
        surface,
```

The main correction is that this:

Python

Run

```
def _random_obstacle(self):
    """
    ...
    """
```

now has the **entire method body properly indented**, and `def update(self)` is back at the same indentation level as `_random_obstacle()`.

Your **120-pixel minimum-distance rule is also preserved**.

```
C:\Users\91895\OneDrive\Documents\06_coin_collector-
main\coin-collector>python main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.13.0)
Hello from the pygame community.
https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File
"C:\Users\91895\OneDrive\Documents\06_coin_collector-
main\coin-collector\main.py", line 63, in <module>
    main()
    ~~~~^^
```

Show more

Yep — the error is because the `handle_input()` method is **missing** from the `GameEngine` file you pasted. `main.py` calls:

Python

Run

```
engine.handle_input(keys)
```

but `GameEngine` doesn't have that method.

Add this method **between** `_random_obstacle()` **and** `update()`:

Python

Run

```
def handle_input(self, keys_pressed):  
    """Handle player movement using the arrow keys."""  
  
    if self.game_over:  
        return  
  
    dx = 0  
    dy = 0  
  
    if keys_pressed[pygame.K_UP]:  
        dy -= self.player.speed  
  
    if keys_pressed[pygame.K_DOWN]:  
        dy += self.player.speed  
  
    if keys_pressed[pygame.K_LEFT]:  
        dx -= self.player.speed  
  
    if keys_pressed[pygame.K_RIGHT]:  
        dx += self.player.speed  
  
    self.player.move(  
        dx,  
        dy,  
        WIDTH,  
        HEIGHT  
    )
```

So this section of your file should look like:

Python

Run

```
def _random_obstacle(self):  
    """  
    Create an obstacle at least 120 pixels away  
    from the player's starting position.  
    """  
  
    player_x = self.player.x  
    player_y = self.player.y  
  
    minimum_distance = 120  
  
    for _ in range(100):  
        x = random.randint(  
            30,  
            WIDTH - 30  
        )
```

```

        y = random.randint(
            70,
            HEIGHT - 30
        )

        dx = x - player_x
        dy = y - player_y

        distance = (dx * dx + dy * dy) ** 0.5

        if distance >= minimum_distance:
            return Obstacle(
                x=x,
                y=y,
                radius=16,
                dx=random.choice([-3, -2, 2, 3]),
                dy=random.choice([-3, -2, 2, 3]),
                color=(220, 60, 60)
            )

    return Obstacle(
        x=30,
        y=70,
        radius=16,
        dx=random.choice([-3, -2, 2, 3]),
        dy=random.choice([-3, -2, 2, 3]),
        color=(220, 60, 60)
    )

def handle_input(self, keys_pressed):
    """Handle player movement using the arrow keys."""

    if self.game_over:
        return

    dx = 0
    dy = 0

    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed

    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed

    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed

    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed

    self.player.move(
        dx,

```

```

        dy,
        WIDTH,
        HEIGHT
    )

    def update(self):
        if self.game_over:
            return

```

Then run:

```
python main.py
```

The important thing is that `handle_input()` must have **4 spaces of indentation**, exactly like `update()`, `_random_coin()`, and `_random_obstacle()`.

Your `main.py` is already correct if it contains:

Python

Run

```

keys = pygame.key.get_pressed()

engine.handle_input(keys)
engine.update()
engine.draw(screen, font)

```

So don't change `main.py`.

give full file content

Here is the **complete corrected** `game/game_engine.py`. This includes everything we've added so far: 3 coin types, 6 always-visible coins, replacement spawning away from obstacles, bouncing obstacles, 120-pixel safe obstacle spawning, 3 lives, invulnerability, 30-second timer, frozen game-over timer, restart support, and the missing `handle_input()`.

Python

Run

```

"""
GameEngine: owns the player, coins, and moving obstacles.

Coin types:
- Bronze: 1 point
- Silver: 3 points

```



```

        x=WIDTH / 2,
        y=HEIGHT / 2
    )

    # -----
    # Score and lives
    # -----

    self.score = 0
    self.lives = STARTING_LIVES

    # -----
    # Invulnerability
    # -----

    self.invulnerable_until = 0.0

    # -----
    # Timer
    # -----

    self.start_time = time.monotonic()
    self.remaining_time = ROUND_TIME

    self.game_over = False

    # -----
    # Obstacles
    # -----

    # Create obstacles first so that
    # coins can avoid them.
    self.obstacles = [
        self._random_obstacle()
        for _ in range(NUM_OBSTACLES)
    ]

    # -----
    # Coins
    # -----

    # Always start with exactly 6 coins.
    self.coins = [
        self._random_coin()
        for _ in range(NUM_COINS)
    ]

def _random_coin(self):
    """
    Create a random coin that does not overlap
    any obstacle.
    """

```



```

        [-3, -2, 2, 3]
    ),
    dy=random.choice(
        [-3, -2, 2, 3]
    ),
    color=(220, 60, 60)
)

# Fallback position.
return Obstacle(
    x=30,
    y=70,
    radius=16,
    dx=random.choice(
        [-3, -2, 2, 3]
    ),
    dy=random.choice(
        [-3, -2, 2, 3]
    ),
    color=(220, 60, 60)
)

def handle_input(self, keys_pressed):
    """Handle player movement using the arrow keys."""

    # Don't allow movement after game over.
    if self.game_over:
        return

    dx = 0
    dy = 0

    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed

    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed

    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed

    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed

    self.player.move(
        dx,
        dy,
        WIDTH,
        HEIGHT
    )

def update(self):

```

```

"""Update the game state."""

# Don't update anything after game over.
if self.game_over:
    return

current_time = time.monotonic()

# -----
# Countdown timer
# -----

elapsed_time = (
    current_time -
    self.start_time
)

self.remaining_time = max(
    0.0,
    ROUND_TIME - elapsed_time
)

# Timer reached zero.
if self.remaining_time <= 0:
    self.remaining_time = 0.0
    self.game_over = True
    return

# -----
# Update obstacles
# -----

for obstacle in self.obstacles:
    obstacle.update(
        WIDTH,
        HEIGHT
    )

# -----
# Collect coins
# -----

collected = check_collection(
    self.player,
    self.coins
)

# Award each collected coin exactly once.
for coin in collected:
    self.score += coin.value

if collected:

```

```

        # Remove collected coins.
        self.coins = [
            coin
            for coin in self.coins
            if coin not in collected
        ]

        # Spawn one replacement for every
        # coin that was collected.
        for _ in collected:
            self.coins.append(
                self._random_coin()
            )

# -----
# Obstacle collision
# -----

if current_time >= (
    self.invulnerable_until
):
    player_rect = self.player.get_rect()

    for obstacle in self.obstacles:

        if player_rect.colliderect(
            obstacle.get_rect()
        ):
            self.lives -= 1

            # One second of invulnerability.
            self.invulnerable_until = (
                current_time +
                INVULNERABILITY_TIME
            )

            # Only lose one life per
            # collision check.
            break

# -----
# Lives game over
# -----

if self.lives <= 0:

    self.lives = 0

    # Freeze the timer at the exact
    # moment the final life is lost.
    self.remaining_time = max(

```



```

# -----

renderer.draw_text(
    surface,
    font,
    f"Score: {self.score}",
    (10, 10)
)

# -----
# Lives
# -----

renderer.draw_text(
    surface,
    font,
    f"Lives: {self.lives}",
    (10, 40)
)

# -----
# Timer
# -----

renderer.draw_text(
    surface,
    font,
    f"Time: {int(remaining_time + 0.999)}",
    (10, 70)
)

# -----
# Game-over screen
# -----

if self.game_over:
    renderer.draw_game_over(
        surface

```

Your `main.py` can continue using:

Python

Run

```

keys = pygame.key.get_pressed()
engine.handle_input(keys)
engine.update()
engine.draw(screen, font)

```

The `AttributeError` should now be gone because `handle_input()` is correctly defined at the same level as `update()`, `draw()`, and the other `GameEngine` methods.

Make the obstacle collision circle-based to match how obstacles are drawn, make the player blink during the invulnerability period, and show the reason on the game-over screen ('Time's up!' or 'Out of lives!' give full updated file

Replace `game/game_engine.py` with this full version. It includes all previous features plus:

- Circle-to-circle obstacle collision.
- Player blinks while invulnerable.
- Game-over reason is stored as either `"Time's up!"` or `"Out of lives!"`.
- The reason is passed to the renderer.

Python

Run

```
"""
GameEngine: owns the player, coins, and moving obstacles.

Coin types:
- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

There are always 6 coins on screen.
When a coin is collected, a new random coin is spawned
at a position that does not overlap an obstacle.

Each coin is scored exactly once.

The player starts with 3 lives and has 30 seconds per round.
The round ends when the timer reaches zero or lives reach zero.
Press R to restart the round.

During the invulnerability period after being hit,
the player blinks to indicate temporary immunity.
"""

import random
import time
import pygame

from game.player import Player
from game.coin import Coin
from game.obstacle import Obstacle
from game.collection import check_collection
```



```

self.start_time = time.monotonic()
self.remaining_time = ROUND_TIME

self.game_over = False

# Stores why the round ended.
self.game_over_reason = ""

# -----
# Obstacles
# -----

self.obstacles = [
    self._random_obstacle()
    for _ in range(NUM_OBSTACLES)
]

# -----
# Coins
# -----

self.coins = [
    self._random_coin()
    for _ in range(NUM_COINS)
]

def _random_coin(self):
    """
    Create a random coin that does not overlap
    any obstacle.
    """

    for _ in range(100):
        x = random.randint(
            COIN_RADIUS,
            WIDTH - COIN_RADIUS
        )

        y = random.randint(
            70 + COIN_RADIUS,
            HEIGHT - COIN_RADIUS
        )

        value, color = random.choice(
            COIN_TYPES
        )

        candidate = Coin(
            x=x,
            y=y,
            radius=COIN_RADIUS,
            value=value,

```



```

        """Handle player movement using arrow keys."""

        if self.game_over:
            return

        dx = 0
        dy = 0

        if keys_pressed[pygame.K_UP]:
            dy -= self.player.speed

        if keys_pressed[pygame.K_DOWN]:
            dy += self.player.speed

        if keys_pressed[pygame.K_LEFT]:
            dx -= self.player.speed

        if keys_pressed[pygame.K_RIGHT]:
            dx += self.player.speed

        self.player.move(
            dx,
            dy,
            WIDTH,
            HEIGHT
        )

def update(self):
    """Update the game state."""

    if self.game_over:
        return

    current_time = time.monotonic()

    # -----
    # Countdown timer
    # -----

    elapsed_time = (
        current_time -
        self.start_time
    )

    self.remaining_time = max(
        0.0,
        ROUND_TIME - elapsed_time
    )

    if self.remaining_time <= 0:
        self.remaining_time = 0.0
        self.game_over = True

```

```

        self.game_over_reason = "Time's up!"
        return

# -----
# Update obstacles
# -----

for obstacle in self.obstacles:
    obstacle.update(
        WIDTH,
        HEIGHT
    )

# -----
# Collect coins
# -----

collected = check_collection(
    self.player,
    self.coins
)

for coin in collected:
    self.score += coin.value

if collected:
    # Remove collected coins.
    self.coins = [
        coin
        for coin in self.coins
        if coin not in collected
    ]

    # Spawn replacements so there are
    # always 6 coins.
    for _ in collected:
        self.coins.append(
            self._random_coin()
        )

# -----
# Obstacle collision
# -----

if current_time >= self.invulnerable_until:

    # Circle-to-circle collision.
    player_x = self.player.x
    player_y = self.player.y

    # Use the player's actual radius.
    player_radius = self.player.radius

```

```

        for obstacle in self.obstacles:
            dx = (
                player_x -
                obstacle.x
            )

            dy = (
                player_y -
                obstacle.y
            )

            distance_squared = (
                dx * dx +
                dy * dy
            )

            collision_distance = (
                player_radius +
                obstacle.radius
            )

            if distance_squared <= (
                collision_distance ** 2
            ):
                # Player has been hit.
                self.lives -= 1

                # Start invulnerability.
                self.invulnerable_until = (
                    current_time +
                    INVULNERABILITY_TIME
                )

                break

        # -----
        # Lives game over
        # -----

    if self.lives <= 0:
        self.lives = 0

        # Freeze the timer.
        self.remaining_time = max(
            0.0,
            ROUND_TIME - (
                current_time -
                self.start_time
            )
        )
    )

```

```

        self.game_over = True
        self.game_over_reason = "Out of lives!"

def get_remaining_time(self):
    """
    Return remaining time.

    Once the round ends, return the stored value
    so the timer stays frozen.
    """

    if self.game_over:
        return self.remaining_time

    current_time = time.monotonic()

    elapsed_time = (
        current_time -
        self.start_time
    )

    self.remaining_time = max(
        0.0,
        ROUND_TIME - elapsed_time
    )

    return self.remaining_time

def _is_player_invulnerable(self):
    """Return True while the player is invulnerable."""

    if self.game_over:
        return False

    return (
        time.monotonic() <
        self.invulnerable_until
    )

def draw(self, surface, font):
    """Draw the complete game scene."""

    from game import renderer

    # Determine whether the player should
    # currently be visible.
    player_visible = True

    if self._is_player_invulnerable():
        # Blink approximately 10 times per second.
        blink_phase = int(
            time.monotonic() * 10

```



```
# Game-over screen
# -----

if self.game_over:
    renderer.draw_game_over(
        surface,
```

Important: because this `game_engine.py` now passes `player_visible` and `game_over_reason` to the renderer, your `game/renderer.py` also needs its corresponding `draw_scene()` and `draw_game_over()` parameters updated.

Hello from the pygame community.
<https://www.pygame.org/contribute.html>

```
C:\Users\91895\OneDrive\Documents\06_coin_collector-
main\coin-collector>python main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.13.0)
Hello from the pygame community.
https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File
"C:\Users\91895\OneDrive\Documents\06_coin_collector-
Show more
```

The error is because your `Player` class doesn't have a `radius` attribute. Since your player is drawn as a rectangle, we should use the player's rectangle to derive a collision radius instead of assuming `self.player.radius` exists.

Replace `game/game_engine.py` with this corrected full file. The circle collision uses the player's rectangle's center and half of its smaller dimension as the player collision radius.

Python

Run

```
"""
GameEngine: owns the player, coins, and moving obstacles.

Coin types:
- Bronze: 1 point
- Silver: 3 points
- Gold: 5 points

There are always 6 coins on screen.
When a coin is collected, a new random coin is spawned
```



```

self.lives = STARTING_LIVES

self.invulnerable_until = 0.0

self.start_time = time.monotonic()
self.remaining_time = ROUND_TIME

self.game_over = False
self.game_over_reason = ""

# Create obstacles first so that coins
# can avoid them when spawning.
self.obstacles = [
    self._random_obstacle()
    for _ in range(NUM_OBSTACLES)
]

# Always start with exactly 6 coins.
self.coins = [
    self._random_coin()
    for _ in range(NUM_COINS)
]

def _random_coin(self):
    """
    Create a random coin that does not overlap
    any obstacle.
    """

    for _ in range(100):
        x = random.randint(
            COIN_RADIUS,
            WIDTH - COIN_RADIUS
        )

        y = random.randint(
            70 + COIN_RADIUS,
            HEIGHT - COIN_RADIUS
        )

        value, color = random.choice(
            COIN_TYPES
        )

        candidate = Coin(
            x=x,
            y=y,
            radius=COIN_RADIUS,
            value=value,
            color=color
        )

```



```

        return

    dx = 0
    dy = 0

    if keys_pressed[pygame.K_UP]:
        dy -= self.player.speed

    if keys_pressed[pygame.K_DOWN]:
        dy += self.player.speed

    if keys_pressed[pygame.K_LEFT]:
        dx -= self.player.speed

    if keys_pressed[pygame.K_RIGHT]:
        dx += self.player.speed

    self.player.move(
        dx,
        dy,
        WIDTH,
        HEIGHT
    )

def update(self):
    """Update the game state."""

    if self.game_over:
        return

    current_time = time.monotonic()

    # -----
    # Countdown timer
    # -----

    elapsed_time = (
        current_time -
        self.start_time
    )

    self.remaining_time = max(
        0.0,
        ROUND_TIME - elapsed_time
    )

    if self.remaining_time <= 0:
        self.remaining_time = 0.0
        self.game_over = True
        self.game_over_reason = "Time's up!"
        return

```

```

# -----
# Update obstacles
# -----

for obstacle in self.obstacles:
    obstacle.update(
        WIDTH,
        HEIGHT
    )

# -----
# Collect coins
# -----

collected = check_collection(
    self.player,
    self.coins
)

for coin in collected:
    self.score += coin.value

if collected:
    # Remove collected coins.
    self.coins = [
        coin
        for coin in self.coins
        if coin not in collected
    ]

    # Spawn one replacement for every
    # coin collected.
    for _ in collected:
        self.coins.append(
            self._random_coin()
        )

# -----
# Obstacle collision
# -----

if current_time >= self.invulnerable_until:

    # The player is drawn as a rectangle.
    # Use the rectangle's center and derive
    # a collision radius from its size.
    player_rect = self.player.get_rect()

    player_x = player_rect.centerx
    player_y = player_rect.centery

    player_radius = min(

```

```

        player_rect.width,
        player_rect.height
    ) / 2

    for obstacle in self.obstacles:

        dx = (
            player_x -
            obstacle.x
        )

        dy = (
            player_y -
            obstacle.y
        )

        distance_squared = (
            dx * dx +
            dy * dy
        )

        collision_distance = (
            player_radius +
            obstacle.radius
        )

        # Circle-to-circle collision.
        if distance_squared <= (
            collision_distance ** 2
        ):
            self.lives -= 1

            # Start the invulnerability period.
            self.invulnerable_until = (
                current_time +
                INVULNERABILITY_TIME
            )

            break

    # -----
    # Lives game over
    # -----

    if self.lives <= 0:
        self.lives = 0

        # Freeze the timer at the moment
        # the final life is lost.
        self.remaining_time = max(
            0.0,
            ROUND_TIME - (

```

```

        current_time =
            self.start_time
    )
)

self.game_over = True
self.game_over_reason = "Out of lives!"

def get_remaining_time(self):
    """
    Return remaining time.

    Once the round ends, return the stored value
    so the timer stays frozen.
    """

    if self.game_over:
        return self.remaining_time

    current_time = time.monotonic()

    elapsed_time = (
        current_time -
        self.start_time
    )

    self.remaining_time = max(
        0.0,
        ROUND_TIME - elapsed_time
    )

    return self.remaining_time

def _is_player_invulnerable(self):
    """Return True while the player is invulnerable."""

    if self.game_over:
        return False

    return (
        time.monotonic() <
        self.invulnerable_until
    )

def draw(self, surface, font):
    """Draw the complete game scene."""

    from game import renderer

    # Blink the player during invulnerability.
    player_visible = True

```

```

if self._is_player_invulnerable():
    blink_phase = int(
        time.monotonic() * 10
    )

    player_visible = (
        blink_phase % 2 == 0
    )

renderer.draw_scene(
    surface,
    self.player,
    self.coins,
    self.obstacles,
    player_visible=player_visible
)

remaining_time = (
    self.get_remaining_time()
)

# -----
# Score
# -----

renderer.draw_text(
    surface,
    font,
    f"Score: {self.score}",
    (10, 10)
)

# -----
# Lives
# -----

renderer.draw_text(
    surface,
    font,
    f"Lives: {self.lives}",
    (10, 40)
)

# -----
# Timer
# -----

renderer.draw_text(
    surface,
    font,
    f"Time: {int(remaining_time + 0.999)}",
    (10, 70)
)

```

```

)

# -----
# Game-over screen
# -----

if self.game_over:
    renderer.draw_game_over(
        surface,

```

The specific error is fixed here:

Python

Run

```

player_rect = self.player.get_rect()

player_x = player_rect.centerx
player_y = player_rect.centery

player_radius = min(
    player_rect.width,
    player_rect.height
) / 2

```

So we don't assume `Player` has a `radius` attribute.

```

C:\Users\91895\OneDrive\Documents\06_coin_collector-
main\coin-collector>python main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.13.0)
Hello from the pygame community.
https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File
"C:\Users\91895\OneDrive\Documents\06_coin_collector-
main\coin-collector\main.py", line 63, in <module>
    main()
Show more

```

Correct — this time the problem is in `game/renderer.py`, not `game_engine.py`.

Your `game_engine.py` now calls:

Python

Run

```

renderer.draw_scene(
    surface,
    self.player,
    self.coins,
    self.obstacles,
    player_visible=player_visible
)

```

but your existing `draw_scene()` doesn't accept `player_visible`.

Since you asked for the full updated file, replace `game/renderer.py` completely with this:

Python

Run

```

"""
Renderer: draws the game scene, UI, and game-over screen.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (35, 45, 35)
COLOR_PLAYER = (80, 180, 255)
COLOR_TEXT = (255, 255, 255)

def draw_scene(
    surface,
    player,
    coins,
    obstacles=None,
    player_visible=True
):
    """
    Draw the complete game scene.

    player_visible is used to make the player blink
    during the invulnerability period.
    """

    surface.fill(COLOR_BG)

    # -----
    # Draw coins
    # -----

    for coin in coins:
        pygame.draw.circle(

```

```

        surface,
        coin.color,
        (int(coin.x), int(coin.y)),
        coin.radius
    )

# -----
# Draw obstacles
# -----

if obstacles is not None:
    for obstacle in obstacles:
        pygame.draw.circle(
            surface,
            obstacle.color,
            (
                int(obstacle.x),
                int(obstacle.y)
            ),
            obstacle.radius
        )

# -----
# Draw player
# -----

# During invulnerability, the GameEngine
# alternates this between True and False.
if player_visible:
    pygame.draw.rect(
        surface,
        COLOR_PLAYER,
        player.get_rect(),
        border_radius=4
    )

def draw_text(
    surface,
    font,
    text,
    pos,
    color=COLOR_TEXT
):
    """Draw text on the screen."""

    surface.blit(
        font.render(
            text,
            True,
            color
        ),

```

```

        pos
    )

def draw_game_over(
    surface,
    font,
    score,
    reason
):
    """
    Draw the game-over screen with the final score
    and the reason the round ended.
    """

    # -----
    # Dark transparent overlay
    # -----

    overlay = pygame.Surface(
        surface.get_size(),
        pygame.SRCALPHA
    )

    overlay.fill(
        (0, 0, 0, 170)
    )

    surface.blit(
        overlay,
        (0, 0)
    )

    # -----
    # Fonts
    # -----

    game_over_font = pygame.font.SysFont(
        "consolas",
        42,
        bold=True
    )

    reason_font = pygame.font.SysFont(
        "consolas",
        26,
        bold=True
    )

    score_font = pygame.font.SysFont(
        "consolas",
        28
    )

```



```

    )
)

surface.blit(
    score_surface,
    score_surface.get_rect(
        center=(center_x, 265)
    )
)

surface.blit(
    restart_surface,
    restart_surface.get_rect(
        center=(center_x, 320)
    )
)

```

One more important thing

Your current `game_engine.py` expects these two renderer signatures:

Python

Run

```

draw_scene(
    surface,
    player,
    coins,
    obstacles,
    player_visible=player_visible
)

```

and:

Python

Run

```

draw_game_over(
    surface,
    font,
    self.score,
    self.game_over_reason
)

```

The file above supports **both**, so the error you're seeing should be resolved.

After replacing `renderer.py`, run:

```
python main.py
```

You should now get:

-  Player

-  Bronze = 1
-  Silver = 3
-  Gold = 5
-  Moving circular obstacles
-  3 lives
-  30-second timer
- Player **blinks for 1 second after being hit**
- Circle-based player/obstacle collision
- **"Time's up!"** when timer reaches zero
- **"Out of lives!"** when lives reach zero
- Final score
-  to restart

ChatGPT can make mistakes. Check important info.